

Intro — Graph Theory in Python: Garbage Truck Route Planning

 medium.com/@davidfliang/intro-graph-theory-in-python-garbage-truck-route-planning-6c2df3770bcb

David Liang

March 20, 2026

Member-only story

The Closed Tour Problem by Example: Route Optimization

The Problem Statement

You are planning the most efficient route for a garbage truck that must collect trash from every street in the **Green Valley neighborhood**. The truck starts and ends at the **Depot (D)**.

The street network is given below (all streets are **two-way**), with travel times in minutes between intersections:

A — B: 3 min B — C: 4 min B — D: 5 min B — E: 2 min C — F: 6 min D — E: 4 min D — G: 4 min D —

Question:

Determine the **minimum total time** needed for the truck to travel each street **at least once** and return to the depot.

Your answer must include:

1. The in minutes.
2. The () taken to achieve this minimum time, starting and ending at D (thus, forming a).

For reference, a graph representation of the street network is provided for better understanding:

Figure 1 — The Street Network

Introduction

The **Garbage Collection Route Problem**, also known as the **Chinese Postman Problem** for undirected graphs, involves efficiently traversing every street in a service area. The goal is to find the **shortest closed route** that covers every road **at least once** and returns to the **depot**, ensuring all areas are serviced with **minimal travel**.

In this article, we will start by exploring **Eulerian graphs** and **circuits**, which form the foundation for finding such routes efficiently. We will use a **simple example graph** to illustrate the concepts before moving on to algorithms that construct the **optimal tour**.

Section 1 — The Eulerian Circuit

Before discussing the Eulerian circuit, let's take a look at the basic underlying concept: **the Eulerian graph**.

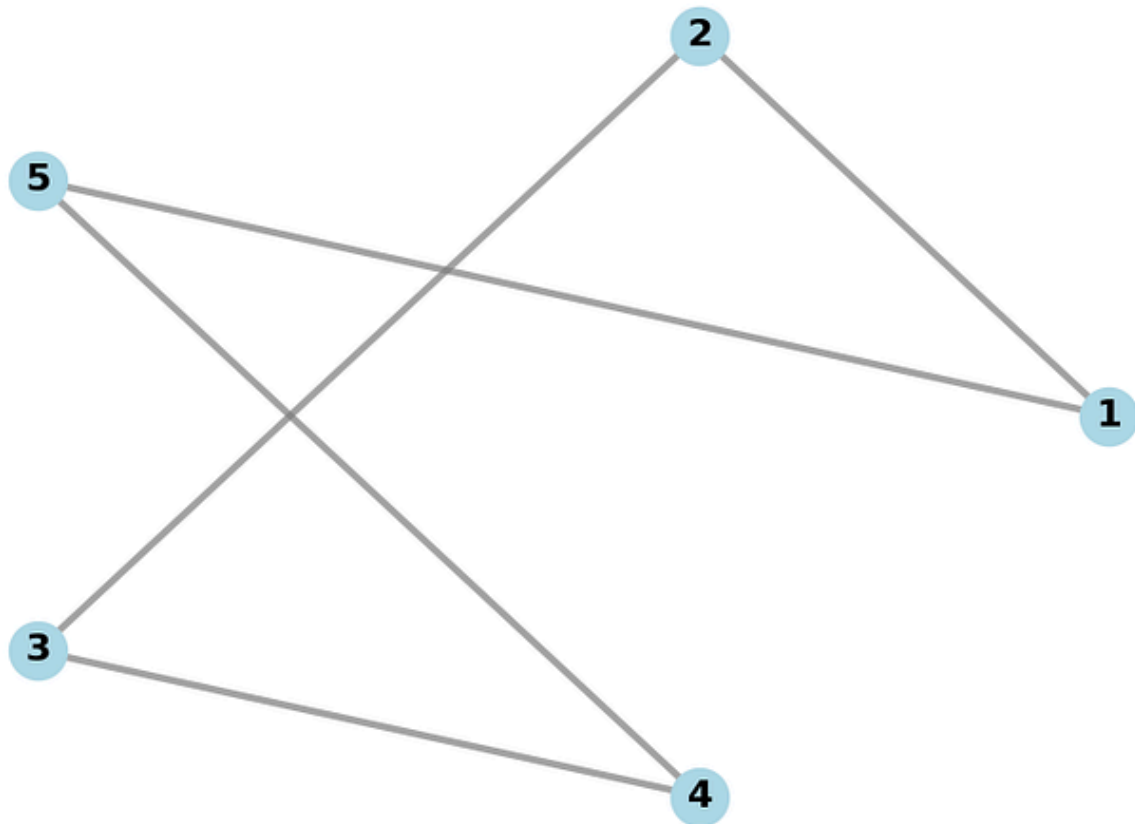
1.1 The Eulerian Graph

An **Eulerian graph** is a graph in which every node has **an even degree** and the **graph is connected**. Such graphs allow for traversing every edge exactly once without getting stuck.

Below is the graph we'll use throughout the example. It consists of **five nodes** arranged in a simple loop, with each node connected to **exactly two others**. Because every node has an even degree and the graph is connected, it forms a valid **Eulerian graph**, making it a clean and minimal structure for illustrating the concepts in this section.

The Graph Structure:

Graph Structure (Nodes 1-5)



The Adjacency List:

```
graph = { 1 : [ 2, 5 ], 2 : [ 1, 3, 4 ], 3 : [ 2, 4, 5 ], 4 : [ 2, 3, 5 ], 5 : [ 1, 3, 4 ] }
```

This adjacency list describes a perfect cycle: starting from any node, you can traverse all edges exactly once and return to the beginning, a textbook **Eulerian setup**.

1.2 The Eulerian Circuit

Eulerian Circuit: A **closed path** that traverses **each edge exactly once** and returns to the **start**. For this to exist in an undirected graph:

- The graph must be connected.
- Every vertex must have even degree (number of edges connected to it).

also known as: **Eulerian cycle**.

For our simple **Eulerian graph**, such a path exists, and this path is the **Eulerian circuit** it contains.

The **Eulerian circuit** can also be expressed in **text notation**, showing the sequence of nodes in the path:

```
| Eulerian Circuit: 1 -> 2 -> 3 -> 4 -> 5 -> 1
```

1.3 How to Find the Eulerian Circuit

Knowing that an Eulerian circuit exists is just the first step. The real challenge is **systematically** finding the **actual path**. We need a systematic approach to traverse the graph while ensuring we use each edge **exactly once** and return to our starting point.

For our small-scale problem, a simple **DFS approach** is sufficient to find the **Eulerian circuit** without needing **specialized algorithms**. The graph's modest size makes a **generic search method** both practical and efficient.

DFS:

```
graph = { 1: [2, 5], 2: [1, 3], 3: [2, 4], 4: [3, 5], 5: [1, 4]} () : path :
```

Output:

```
Graph degrees: {1: 2, 2: 2, 3: 2, 4: 2, 5: 2}All vertices have even degree: TrueEulerian circuit exists: True
```

Result:

```
1 -> 2 -> 3 -> 4 -> 5 -> 1
```

1.4 Eulerian Circuit Visualization

We can use Python to create the **visualization chart** for the Eulerian Circuit.

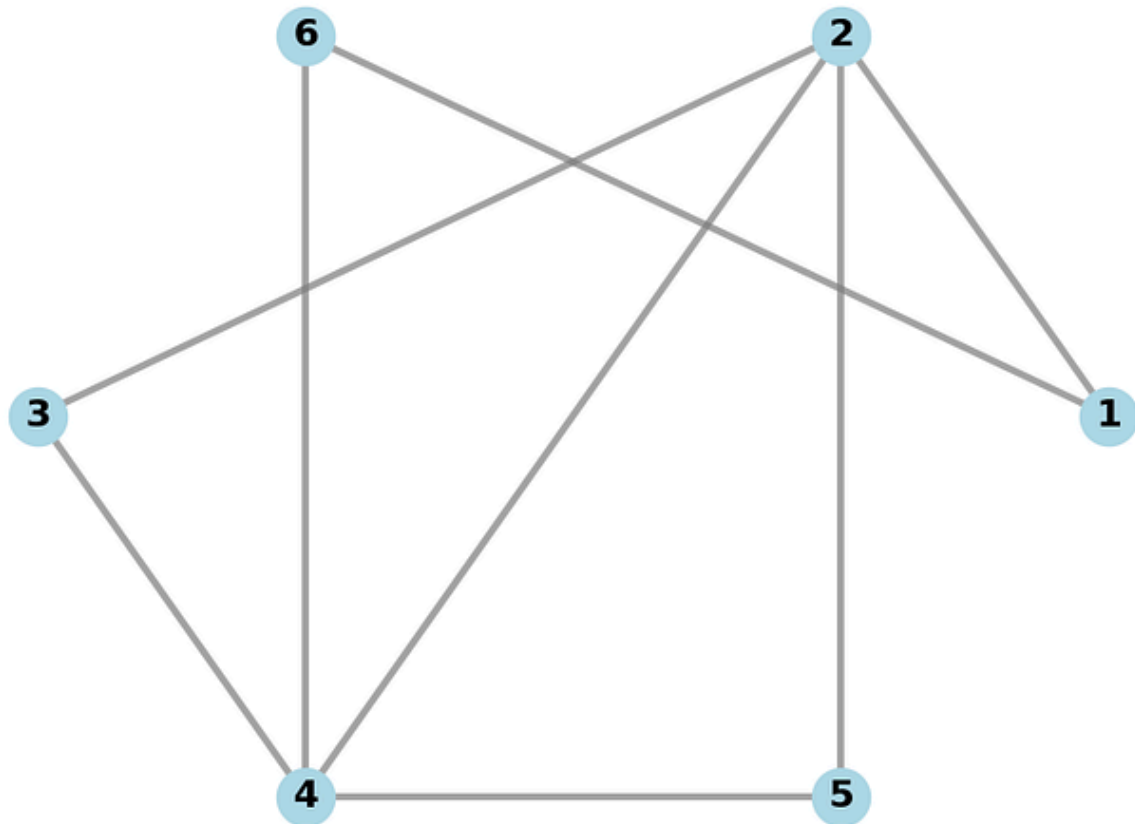
```
import networkx as nx
import matplotlib.pyplot as plt
graph = { 1: [2, 5], 2: [1, 3], 3: [2, 4], 4: [3, 5], 5: [1, 4]}
nx.draw(graph)
```

Output:

1.5 Graph with Degree Variation (2 and 4)

Of course, the degree of each node in the **Eulerian graph** can be any **even number**. For example, we have another **Eulerian graph** with **6 nodes** like this:

Graph Structure (Nodes 1-6)



In the chart above, we can see that **node 2** and **node 4** have a **degree of 4**, meaning they have **4 edges** going through the node.

Again, we can use a **DFS finder** to find the **Eulerian circuit** in the graph.

```
graph = { 1: [2, 4], 2: [1, 3, 4, 5], 3: [2, 4], 4: [1, 2, 3, 5, 6], 5: [2, 4], 6: [4, 5]}()
```

Output:

Eulerian cycle exists (all vertices have even degree)Eulerian cycle: 1 -> 2 -> 3 -> 4 -> 2 -> 5 -> 4 -> 6 -> 1

The Solution Path:

Eulerian Circuit: 1 -> 2 -> 3 -> 4 -> 2 -> 5 -> 4 -> 6 -> 1

We can also create the visual chart.

The Solution Path Visual:

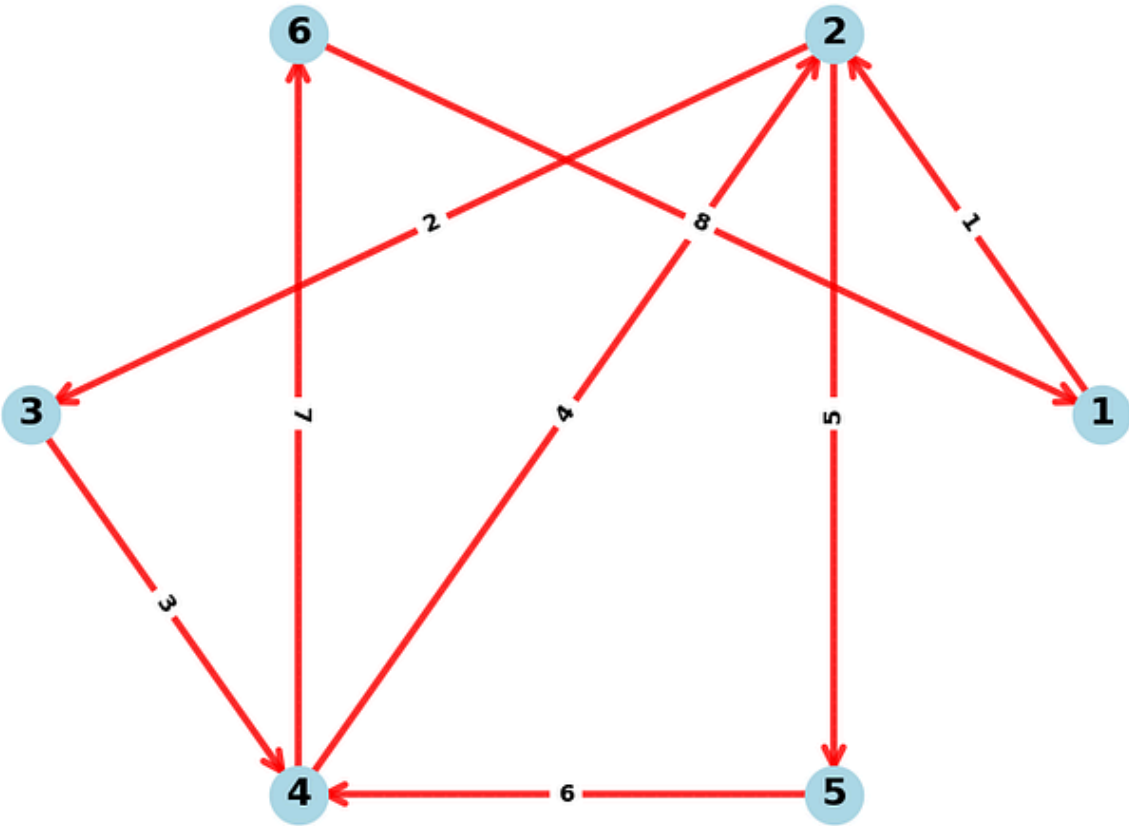
```
networkx nx matplotlib.pyplot plt (): path : path = [start] used_edges = []
```

Output:

Vertex degrees: Vertex 1: degree 2 Vertex 2: degree 4 Vertex 3: degree 2 Vertex 4: degree 4 Vertex 5: degree 2 Vertex 6: degree 2

The Solution Chart:

Eulerian Circuit Path with Step Numbers (DFS Solution)



1.6 Using Python Built-in Method

In fact, we can actually use NetworkX’s **built-in Eulerian circuit** method to solve the same problem.

```
( + *)()(*)is_eulerian = nx.is_eulerian(G) is_eulerian:      ()      source =      (
```

Output:

```
=====METHOD 2: NetworkX Built-in Eulerian Circuit=====
```

Interestingly enough, the solution found by the **built-in method** is **the exact reverse of the path** discovered by our own **DFS finder**.

1.7 Closed Knight's Tour vs. Eulerian Circuit

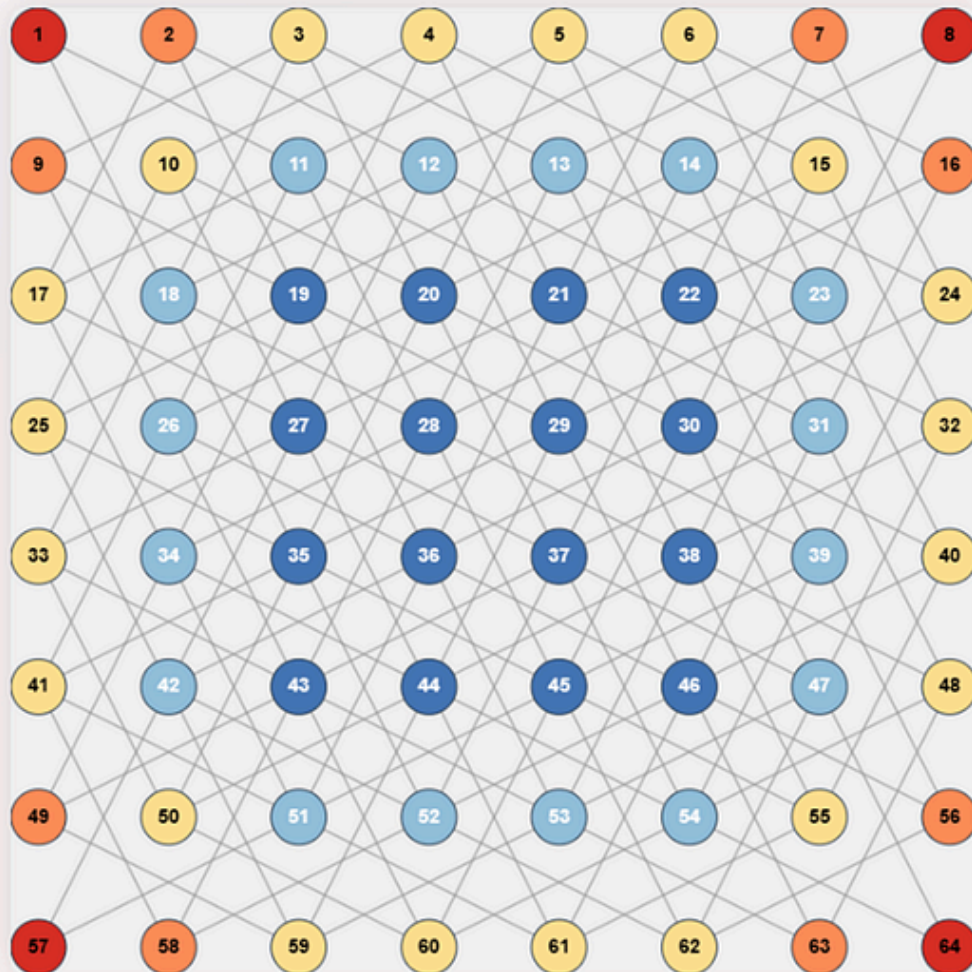
Before moving on to the next section for solving the route planning problem, I'd like to have a quick discussion on **an interesting topic** in graph theory. If you're a regular reader of my posts, you've probably read my article on the **closed Knight's Tour problem**. Basically, the knight needs to travel on the 8×8 chessboard, visit every square exactly once, and return to its starting position.

The concept sounds very similar to an **Eulerian Circuit**. When we represent the **Knight's Tour problem** using a **Knight's Graph** — where each chess square is a node and legal knight moves are edges — the resemblance to an Eulerian Circuit problem becomes even more striking.

Knight's Graph Visualization

This visualization shows all possible knight moves on a chessboard as a node graph. Each node is colored based on the number of legal moves available from that position.

● 2 moves ● 3 moves ● 4 moves ● 6 moves ● 8 moves



Hover over nodes to highlight possible knight moves

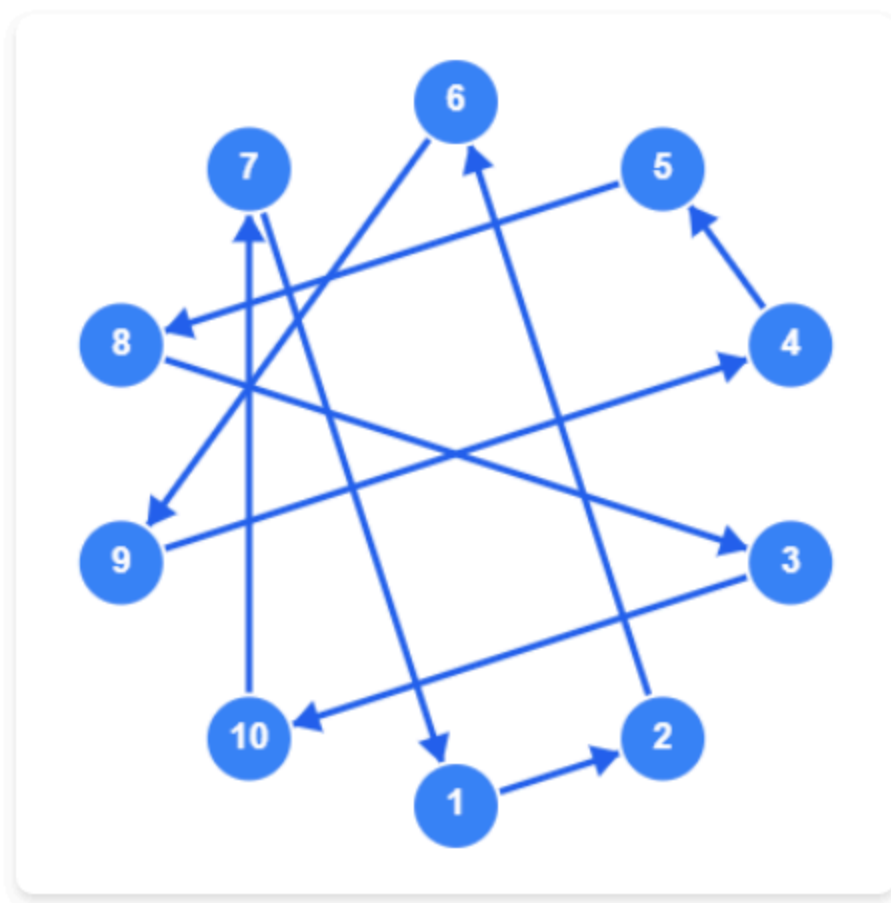
Both involve traversal of connected nodes, which naturally leads to the question: given their apparent **similarities**, how do we actually distinguish between these two fundamental graph theory concepts?

Here's what I discovered:

- A path that visits every (or node exactly once). The degrees of the vertices are not the primary constraint. In fact, a closed knight's tour path only uses out of the in the .
- A path that traverses every exactly once. This requires all vertices to have an even degree.

So, in summary, although both problems involve fully traversing a graph and returning to the **starting position** while **visiting elements** exactly once, the key distinction lies in what is being visited. The **Knight's Tour** requires visiting **every node** once and only once, while the **Eulerian Circuit** requires visiting **every edge** once and only once.

To make this concept even more interesting, you might wonder: do we have situations where **both nodes and edges** are visited exactly once? The answer is yes, and we've discussed this before as well — it's called **Permutation Cycle**.



Cycle Decomposition: (1 2 6 9 4 5 8 3 10 7)

In the graph above, the closed path goes through each node and each edge once and only once.

To be clear, I did not initially plan for this **long story arc** to demonstrate how the **three node graph concepts** are connected. I only discovered that connection while writing this article. Isn't it amazing to see how all these mathematical concepts can be interrelated in such an intricate way?

Section 2 — The Pathfinder Solution

In this section, let's use our knowledge of **Eulerian Circuits** to address the **Garbage Truck Route Planning** problem. We'll start by analyzing the node graph (**Figure 1 below**) representing the street map.

Figure 1 (Repeat)

We can clearly see this graph does not have an **Eulerian Circuit** because it has **four nodes** with **an odd number of edges**:

- A: 1 edge
- D: 5 edges
- G: 3 edges
- I: 3 edges

If the question were asking about finding an **Eulerian Circuit**, the discussion would end here, as none exists. However, the problem statement is different: it asks for the truck to travel each street **at least once** and return to the depot. This **subtle difference** gives us hope.

2.1 Chinese Postman Problem

This problem is actually a direct adaptation of a famous mathematical problem called the "**Chinese Postman Problem**", which was first studied by the Chinese mathematician **Meigu Guan**.

The solution is based on the following **core concepts**:

1. : Since the original graph is not Eulerian, we must modify it to become one. The requirement to traverse each street means we can . In graph terms, this is equivalent to adding a between nodes.
2. : The clever insight is that we don't need to fix all nodes, only the ones. By strategically and adding duplicate edges between each pair, we all degrees to . While existing even-degree nodes might be affected due to adding edges through them, we'll see later why this doesn't impact the overall plan.
3. : Duplicating an edge comes with an additional cost (i.e., the travel time for the repeated segment). Therefore, our goal is to find the set of edge duplications that this total added cost. This requires using a to calculate the most efficient connections between thenodes.
4. : The final is the of the (the time to traverse all original edges once) the from the duplicated edges.

The **base cost** is fixed, as it is simply the sum of all original edge weights. Therefore, **minimizing the added cost** from duplicated edges is the **only factor** that affects the final result.

We can now translate these **core concepts** into an **implementation**.

2.2 The Base Solution

The base solution would only provide the final cost.

Step 1: Build the Graph and Find Odd-Degree Vertices

```
networkx nxedges = [ (1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (3, 4), (3, 5), (4, 5)]
```

Output:

Vertex degrees: A: degree 1 B: degree 4 C: degree 2 D: degree 5 E: degree 4 F: de

We can confirm this step is correct.

Step 2: Compute Shortest Paths Between All Odd Pairs

Since we have four odd-degree nodes, we need to consider all possible ways to pair them.

This gives us $C(4,2) = 6$ distinct pairings.

```
itertools combinationsdist = {} u, v combinations(odds, ): dist_key = ((u, v))
```

Output:

Shortest path A→D: ['A', 'B', 'D'] (length: 8) Shortest path A→G: ['A', 'B', 'D', 'G']

Now, we have the shortest path for each possible combination.

Step 3: Find Optimal Pairing

Next, we need to put the **four elements** into **two groups** in a way that produces the overall shortest path. From a combinatorial perspective, there are $\mathbf{C(4, 2)/2 = 3}$ possible ways to do this. Why divide by 2? Because each grouping is counted twice (e.g., A–D and D–A), so we divide by 2.

```
pairings = [ [(,), (,)], [(,), (,)], [(,), (,)] ]() i, pairing (pairings):
```

Output:

Pairing costs: [('A', 'D'), ('G', 'I')]: 17 [('A', 'G'), ('D', 'I')]: 19 [('A', 'I'), ('D', 'G')]: 18

The best pairing found is [(‘A’, ‘I’), (‘D’, ‘G’)], with a **total added cost of 12**. Since we already have the **final added cost**, we can also answer the first question. But let’s leave that for now and continue adding edges according to the plan.

Step 4: Add Edges Along Shortest Paths

In this step, we begin adding edges to the target nodes. For the pair (D, G), we can add the edge directly since they are already connected. However, for the pair (A, I), we need to add additional edges through intermediate nodes.

```
G_eulerian = nx.MultiGraph(G()) pair optimal_pairing: u, v = pair path = nx.shor
```

Output:

Edges to duplicate along shortest paths: Path A→I: [‘A’, ‘B’, ‘E’, ‘I’] Duplicate: A-B

Now we can see that four edges have been added.

Step 5: Verify all degrees are now even

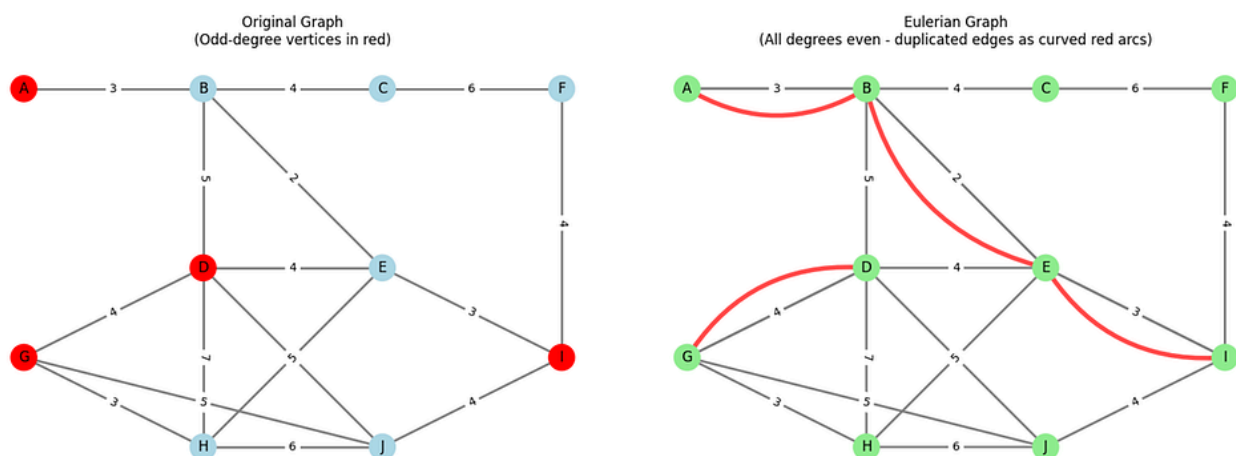
Let’s verify the final graph.

```
() node (G_eulerian.nodes()): deg = G_eulerian.degree(node) ()
```

Output:

Final degrees: A: degree 2 (even) B: degree 6 (even) C: degree 2 (even) D: degree 4 (even)

We can verify the **edge additions** visually by comparing the original and constructed graph side by side.



From the charts above, we can observe that all vertices now have **even degrees**, and the total added cost is $3 + 4 + 2 + 3 = 12$ minutes (the duplicated edges are shown as **curved arcs**).

Note: You may have noticed that **Node B** and **Node E** are also affected when edges are added through them. However, since any path passing through a node always contributes one edge in and one edge out, their degrees remain even. Therefore, the addition does not change their even-degree property.

Step 6: Calculate Total Time

```
total_time = total_edges + min_cost()
```

Output:

Total time: 82

The Answer

In conclusion, the answer to the **first question** posed in the **problem statement** is that the minimum total time required is **82 minutes**.

Python Full Code

This is the complete original code used to calculate the total time.

```
networkx nx itertools combinations, permutationsG = nx.Graph()G.add_edge(, , weight=
```

Output:

Total time: 82

The final total time consists of a base time of **70 minutes** plus a **12-minute** add-on.

2.3 Handshaking Lemma

You may have noticed a potential “**problem**” with the strategy of pairing nodes that have an odd number of edges. While our example had **four such nodes**, what if there were five or any odd number? The definitive answer is that this **will never happen**. The reason is the **Handshaking Lemma**, which proves that in any graph, the number of nodes with an odd number of edges **must be even**.

The Handshaking Lemma states that the sum of the degrees of all vertices in a graph is equal to twice the number of edges, which is always even. Therefore, the number of vertices with odd degrees must be even to make the total sum even.

2.4 The Complete Solution

We successfully constructed a **Eulerian graph** with the addition of edges at minimum cost. Then, we calculated the total minutes required. However, we still need to answer the **second question**, which is to print out a list of nodes as the path.

This is now a **simple task** since we already have a well-constructed **Eulerian graph**. We can use **NetworkX's built-in function** to find the **Eulerian circuit**.

```
source = circuit = (nx.eulerian_circuit(G_eulerian, source=source))path = [source] u, v
```

Output:

```
Final path (closed tour):D -> J -> H -> G -> J -> I -> E -> H -> D -> G -> D -> E -> B
```

Python Full Code:

```
networkx  nx itertools  combinations, permutationsG = nx.Graph()G.add_edge(, , weight=
```

Output:

```
Total edge length: 700dd degree vertices: ['A', 'D', 'G', 'I']Shortest path distances:
```

The Final Sequence:

```
D -> J -> H -> G -> J -> I -> E -> H -> D -> G -> D -> E -> B -> E -> I -> F -> C -> B -> A
-> B -> D
```

The text-based path is hard to follow. Let's visualize it instead.

Section 3 — Visualization

In this section, we'll use the **matplotlib** and **networkx** libraries to build the solution visualization.

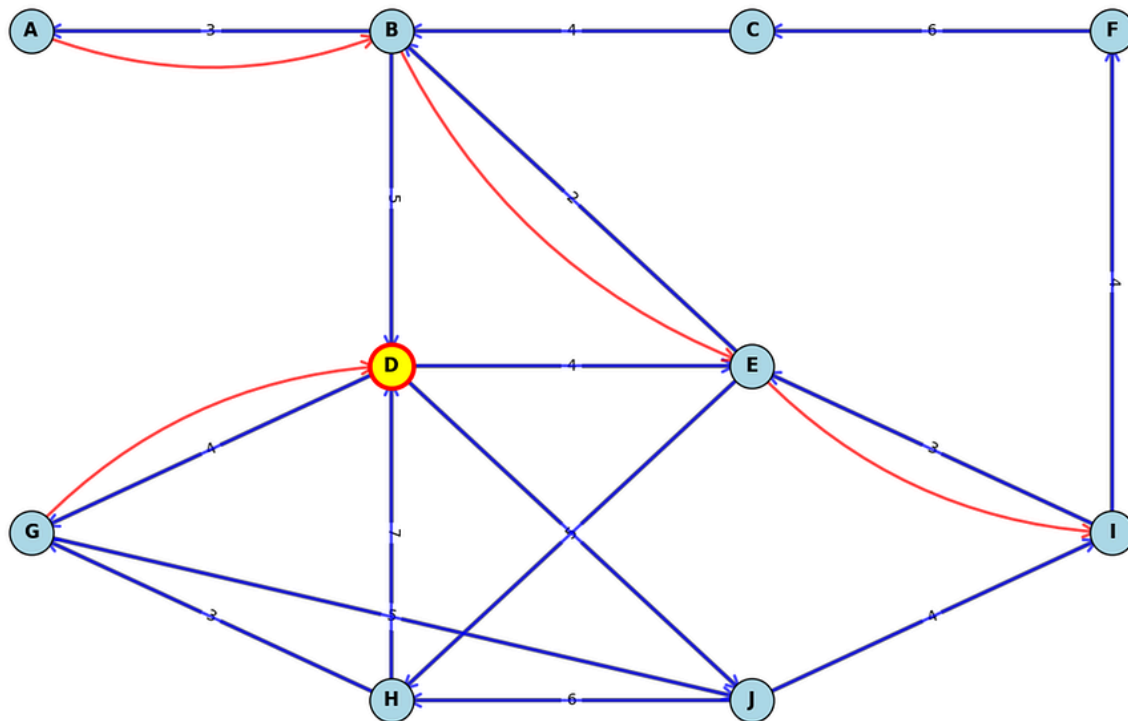
3.1 The Solution Visual

Python Full Code:

```
matplotlib.pyplot  plt networkx  nxG = nx.Graph()edges = [ (, , ), (, , ), (
```

Output:

Garbage Collection Route - Two-Way Streets
 (Depot D highlighted in yellow, First pass in blue arrows, Repeats in red curved arrows)



Final path (closed tour): D -> J -> H -> G -> J -> I -> E -> H -> D -> G -> D -> E -> B -> E -> I -> F -> C -> B -> A -> B -> D

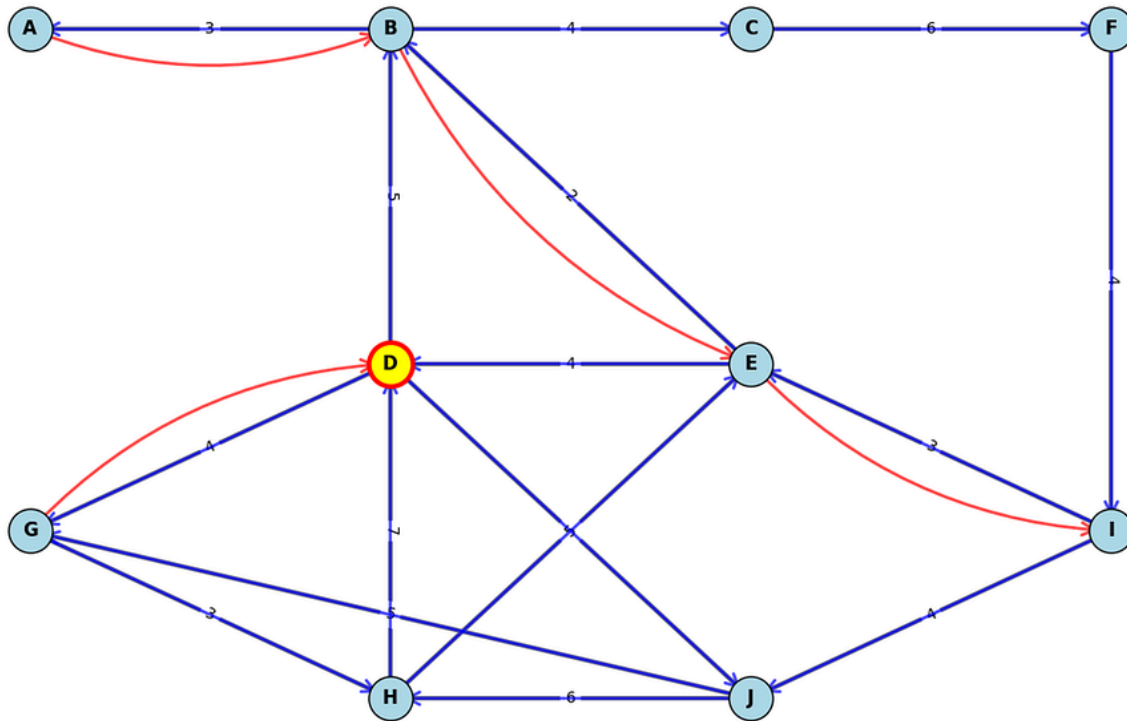
Solution 1

3.2 The Alternative Solution

As you would expect, there are more solutions (in fact, many more). Here is another solution:

D -> B -> A -> B -> C -> F -> I -> E -> B -> E -> D -> G -> H -> D -> J -> H -> E -> I -> J -> G -> D

Garbage Collection Route - Two-Way Streets
 (Depot D highlighted in yellow, First pass in blue arrows, Repeats in red curved arrows)



Final path (closed tour): D -> B -> A -> B -> C -> F -> I -> E -> B -> E -> D -> G -> H -> D -> J -> H -> E -> I -> J -> G -> D

Solution 2

Note: At first glance, the second solution might seem like just the reverse of the first. However, a closer look shows they're not identical.

For example, the reverse of the first path contains $D \rightarrow G \rightarrow D \rightarrow H$, whereas the second solution follows $D \rightarrow G \rightarrow H \rightarrow D$. Not to mention, the endings differ as well: the reverse of the first path finishes through $J \rightarrow D$, while the second path ends through $G \rightarrow D$.

3.3 The Interactive Solution Viewer

I've also created a companion online version of the **Solution Viewer**. It's currently online at this address:

<https://route-gray-one.vercel.app/>

Graph Visualization



Reset View

Solution:

Solution 1

Previous Step

Next Step

The online version is a **HTML/JavaScript** app that displays solution datasets generated by the Python code. It functions as an **interactive solution viewer** only.

Note: Since there are **many solutions** (precisely **30,912**, as found by the **optimized DFS solver** — not shown in this article), only a **small subset** are shown in this online demo.

Summary

In this article, we explore the **Closed Tour Problem**, using a **garbage truck route** as a practical example to demonstrate the application of graph theory. We establish that the core of this challenge is the **Chinese Postman Problem**, where the goal is to find the shortest path traversing **every street at least once**.

The solution centers on **converting** a non-Eulerian graph by identifying odd-degree nodes, finding the minimal-cost pairing between them, and adding the corresponding shortest paths

to **make the graph Eulerian**.

We successfully apply this method to the Green Valley street network, determining **an optimal route** time of **82 minutes** and generating an efficient closed tour that services all streets. Through this hands-on example, we show how abstract graph theory concepts can be turned into practical techniques for solving complex routing puzzles.